CSS

MAKE IT BEAUTIFUL





Turn plain HTML into a **designed** web page.

Session 3 · a very basic introduction

TODAY'S ROADMAP

1 · CSS
What CSS is and how to add it.

2 · SELECTORS
Choose **what** to style.

3 · COLORS & TEXT
Make it readable and pretty.

4 · BOXES
Margin, padding, border.

5 · PRACTICE
Quiz + style your resume.

6 · FRAMEWORKS
A first look at Bootstrap & Tailwind.

SECTION 01

MEET CSS

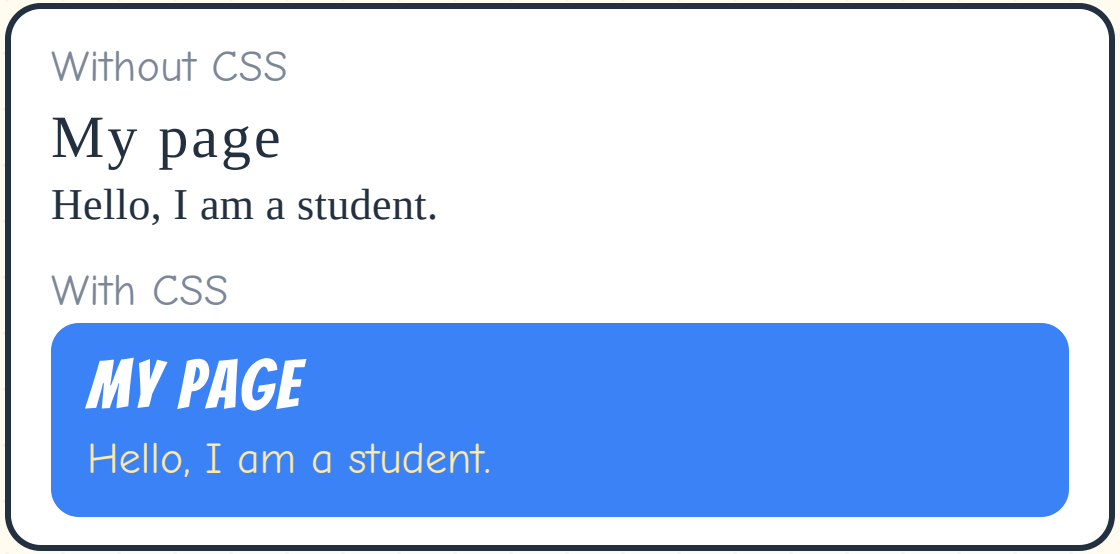
HTML built the walls. CSS is the paint, the furniture and the lighting.



SAME HTML, DIFFERENT CSS

```
<h1>My page</h1>
<p>Hello, I am a student.</p>
```

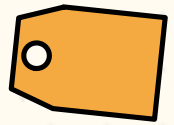
The HTML is **identical** on both sides. Only the CSS changes.



*CSS = Cascading Style Sheets. It describes how HTML elements **look**.*

Anatomy of a CSS rule

```
p {
  color: blue;
  font-size: 20px;
}
```



selector
which elements?

property
what to change

value
the new setting

; and { }
punctuation matters!

> "For **every paragraph**, make the **color** blue and the **font size** 20 pixels."

Three ways to add CSS

1 · Inline

The `style` attribute on one tag.

Avoid

2 · Internal

A `<style>` block in the `head`.

OK for tests

3 · External

A separate `.css` file, linked.

Best

```
<p style="color: red;">Inline</p>

<style>
  p { color: red; }
</style>

<link rel="stylesheet" href="style.css">
```

THE BEST WAY: AN EXTERNAL FILE

index.html

```
<head>
  <meta charset="utf-8">
  <title>My page</title>
  <link rel="stylesheet"
        href="style.css">
</head>
```

style.css

```
body {
  font-family: sans-serif;
  background: #fffbf0;
}
h1 {
  color: #ff6b6b;
}
```

One `.css` file can style **every page** of a website — change it once, everything updates.

SECTION 02

SELECTORS

Before you can style something, you must point at it.



ELEMENT, CLASS AND ID

```css
/* every <p> */
p { color: gray; }

/* anything with class="note" */
.note { background: #fff3c4; }

/* the one element with id="logo" */
#logo { font-size: 40px; }
```

```html
<p>Normal paragraph</p>
<p class="note">Important</p>
<h1 id="logo">MySite</h1>
```

`p`
By **tag name**.

`.note`
By **class**: a dot.
Reusable.

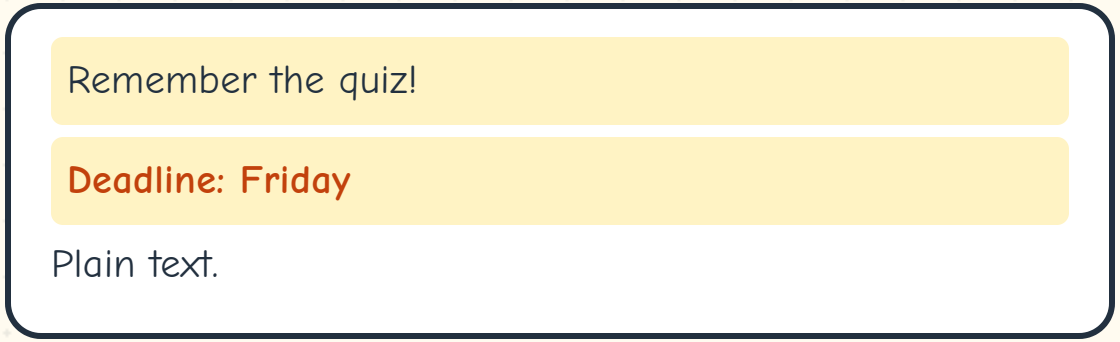
`#logo`
By **id**: a hash.
Unique.

*Use **classes** most of the time. An `id` must appear only **once** per page.*

Classes in action

```html
<p class="note">Remember the quiz!</p>
<p class="note warn">Deadline: Friday</p>
<p>Plain text.</p>
```

```css
.note {
  background: #fff3c4;
  padding: 8px;
}
.warn {
  color: #c2410c;
  font-weight: bold;
}
```



An element can have **several classes**, separated by spaces.

SECTION 03

COLORS & TEXT

The fastest way to make a page look professional.



Colors — four ways to write them

```css
h1 { color: tomato; }                  /* name */
h1 { color: #ff6347; }                 /* hex */
h1 { color: rgb(255, 99, 71); }        /* red, green, blue */
h1 { color: hsl(9, 100%, 64%); }       /* hue, saturation, lightness */
```



Two properties you will use all the time: `color` (the text) and `background-color` (behind it).

TEXT PROPERTIES

```css
p {
  font-family: Arial, sans-serif;
  font-size: 20px;
  font-weight: bold;
  font-style: italic;
  text-align: center;
  line-height: 1.6;
  text-decoration: underline;
  text-transform: uppercase;
}
```

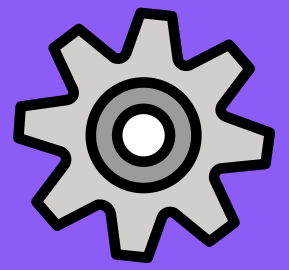
I AM A STYLED PARAGRAPH

Always give a **fallback**: `Arial, sans-serif`. If the first font is missing, the browser tries the next.

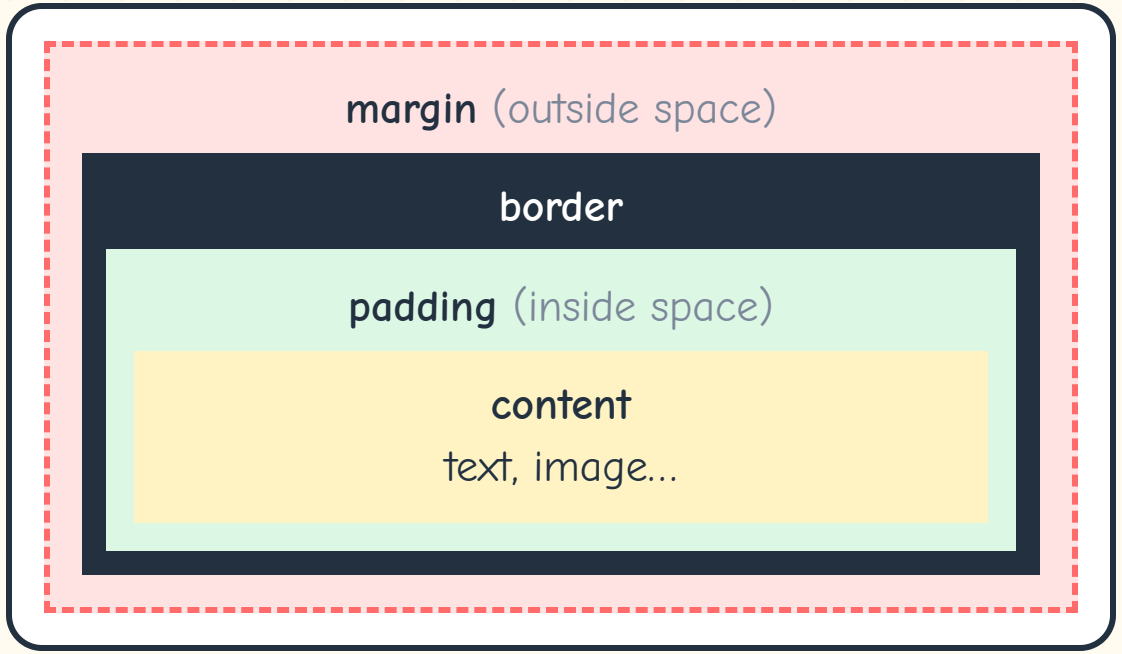
SECTION 04

THE BOX MODEL

Every element on a page is a rectangular box.



FOUR LAYERS



- **content** → the text or image itself
- **padding** → space **inside**, between content and border
- **border** → the edge line
- **margin** → space **outside**, pushing neighbours away

Padding = inside the box. Margin = outside the box.

Margin, Padding, Border

```css
.card {
  padding: 20px;
  border: 3px solid #22303f;
  border-radius: 16px;
  margin: 24px;
}

/* shorthands: top right bottom left */
.card { padding: 10px 20px 10px 20px; }
/* vertical | horizontal */
.card { margin: 0 auto; }
```



`margin: 0 auto` centers a box with a fixed `width`. `border-radius` rounds the corners.

SECTION 05

PRACTICE

A tiny quiz, then style your own resume.



QUICK QUIZ

1. Best way to add CSS to a whole website? **A.** inline `style` **B.** an external `.css` file

2. Which selector targets `class="note"`? **A.** `#note` **B.** `.note` **C.** `note`

3. Space **inside** an element's border is… **A.** margin **B.** padding

4. Which property changes the **text** color? **A.** `color` **B.** `text-color` **C.** `font-color`

Answers: 1 B · 2 B · 3 B · 4 A

YOUR MISSION

Create `style.css`, link it from your `resume.html`, and:

1. Pick a **font** and **2–3 colors**
2. Style headings, text and links (add `:hover`)
3. Give the page a **background color**
4. Put your content in a **centered box** (`max-width`, `margin: 0 auto`, `padding`)
5. Style the **table** with borders and padding

RULES

One external `style.css`. No inline styles.

TIP

Save, refresh, repeat. Press **F12** to try values live.

STARTER STYLESHEET

```
* { box-sizing: border-box; }

body {
  font-family: "Segoe UI", sans-serif;
  line-height: 1.6;
  color: #22303f;
  background: #fffbf0;
  margin: 0;
}

.container { max-width: 800px; margin: 0 auto; padding: 24px; }

h1, h2 { color: #3b82f6; }
a { color: #ff6b6b; }
a:hover { text-decoration: underline; }

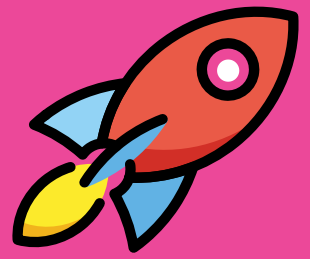
table { border-collapse: collapse; width: 100%; }
th, td { border: 2px solid #22303f; padding: 8px; }
```


Wrap your page content in `<div class="container">` *and grow from here.*

SECTION 06

CSS FRAMEWORKS

Ready-made CSS written by thousands of developers. Here is a first look.



WHAT IS A CSS FRAMEWORK?

A **framework** is a ready-made stylesheet (often with a little JavaScript) that gives you:

- a **grid** and responsive layout tools
- styled **buttons, forms, cards, navbars**…
- a consistent **design system** (colors, spacing, fonts)
- **cross-browser** fixes already done

PROS

Fast to build · consistent look · battle-tested · big communities

CONS

Pages can look alike · extra size · you can forget how CSS works

*A framework is a **shortcut**, not a replacement. You still need the CSS you learned today.*

TWO BIG FAMILIES

COMPONENT FRAMEWORKS

Pre-designed pieces: `btn`, `card`, `navbar`. You assemble them.

Bootstrap · Bulma · Foundation

UTILITY-FIRST FRAMEWORKS

Tiny single-purpose classes: `p-4`, `text-center`, `bg-blue-500`. You compose your own design.

Tailwind CSS · UnoCSS

Component = "here is a button". Utility = "here are the paint tubes".

BOOTSTRAP

COMPONENTS · SINCE 2011

```
<link rel="stylesheet"
 href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
```

```
<div class="container">
  <div class="row">
    <div class="col-md-6">
      <button class="btn btn-primary">Join</button>
    </div>
    <div class="col-md-6">
      <div class="alert alert-success">Hi!</div>
    </div>
  </div>
</div>
```

- **12-column grid**: `container` → `row` → `col-*`
- Ready components: buttons, cards, navbar, forms…
- `col-md-6` = half width from medium screens up
- Best for quick prototypes and school projects

TAILWIND CSS

UTILITY-FIRST · SINCE 2019

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
```

```
<div class="max-w-md mx-auto p-6 bg-white">
  <h1 class="text-2xl font-bold">Hello</h1>
  <button class="px-4 py-2 bg-blue-500
                 text-white rounded
                 hover:bg-blue-600">
    Join
  </button>
</div>
```

- **No pre-made look**: tiny classes, right in the HTML
- `p-6` = padding, `rounded` = round corners, `hover:` = state, `md:` = responsive
- Real projects use a **build step**; the CDN script is for **learning only**
- Best for custom designs and modern apps

WHAT TO REMEMBER

CSS

A rule = selector + properties.
Link one external file.

LOOK

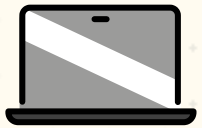
Colors, fonts and the **box model**.

FRAMEWORKS

Bootstrap = components.
Tailwind = utilities.

Learn plain CSS first — every framework is CSS underneath. Next time: ***JavaScript.***

NOW LET'S STYLE


Create `style.css`, link it, and **make it yours**.

Questions? Hands up. Let's build.